

Algorithm 6.2 Simplified version of the back-propagation algorithm for computing the derivatives of $u^{(n)}$ with respect to the variables in the graph. This example is intended to further understanding by showing a simplified case where all variables are scalars, and we wish to compute the derivatives with respect to $u^{(1)}, \dots, u^{(n_i)}$. This simplified version computes the derivatives of all nodes in the graph. The computational cost of this algorithm is proportional to the number of edges in the graph, assuming that the partial derivative associated with each edge requires a constant time. This is of the same order as the number of computations for the forward propagation. Each $\frac{\partial u^{(i)}}{\partial u^{(j)}}$ is a function of the parents $u^{(j)}$ of $u^{(i)}$, thus linking the nodes of the forward graph to those added for the back-propagation graph.

Run forward propagation (algorithm 6.1 for this example) to obtain the activations of the network

Initialize `grad_table`, a data structure that will store the derivatives that have been computed. The entry `grad_table[ $u^{(i)}$ ]` will store the computed value of $\frac{\partial u^{(n)}}{\partial u^{(i)}}$.

`grad_table[ $u^{(n)}$ ]  $\leftarrow$  1`

for $j = n - 1$ down to 1 **do**

The next line computes $\frac{\partial u^{(n)}}{\partial u^{(j)}} = \sum_{i:j \in Pa(u^{(i)})} \frac{\partial u^{(n)}}{\partial u^{(i)}} \frac{\partial u^{(i)}}{\partial u^{(j)}}$ using stored values:

`grad_table[ $u^{(j)}$ ]  $\leftarrow$   $\sum_{i:j \in Pa(u^{(i)})}$  grad_table[ $u^{(i)}$ ]  $\frac{\partial u^{(i)}}{\partial u^{(j)}}$`

end for

return {grad_table[$u^{(i)}$] | $i = 1, \dots, n_i$ }

Back-propagation thus avoids the exponential explosion in repeated subexpressions. However, other algorithms may be able to avoid more subexpressions by performing simplifications on the computational graph, or may be able to conserve memory by recomputing rather than storing some subexpressions. We will revisit these ideas after describing the back-propagation algorithm itself.

6.5.4 Back-Propagation Computation in Fully-Connected MLP

To clarify the above definition of the back-propagation computation, let us consider the specific graph associated with a fully-connected multi-layer MLP.

Algorithm 6.3 first shows the forward propagation, which maps parameters to the supervised loss $L(\hat{\mathbf{y}}, \mathbf{y})$ associated with a single (input, target) training example $(\mathbf{x}, \mathbf{y})$, with $\hat{\mathbf{y}}$ the output of the neural network when $\mathbf{x}$ is provided in input.

Algorithm 6.4 then shows the corresponding computation to be done for

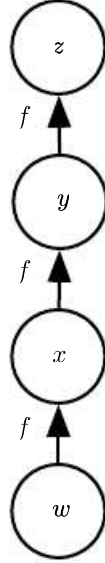


Figure 6.9: A computational graph that results in repeated subexpressions when computing the gradient. Let $w \in \mathbb{R}$ be the input to the graph. We use the same function $f : \mathbb{R} \rightarrow \mathbb{R}$ as the operation that we apply at every step of a chain: $x = f(w)$, $y = f(x)$, $z = f(y)$. To compute $\frac{\partial z}{\partial w}$, we apply equation 6.44 and obtain:

$$\frac{\partial z}{\partial w} \tag{6.50}$$

$$= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w} \tag{6.51}$$

$$= f'(y) f'(x) f'(w) \tag{6.52}$$

$$= f'(f(f(w))) f'(f(w)) f'(w) \tag{6.53}$$

Equation 6.52 suggests an implementation in which we compute the value of $f(w)$ only once and store it in the variable x . This is the approach taken by the back-propagation algorithm. An alternative approach is suggested by equation 6.53, where the subexpression $f(w)$ appears more than once. In the alternative approach, $f(w)$ is recomputed each time it is needed. When the memory required to store the value of these expressions is low, the back-propagation approach of equation 6.52 is clearly preferable because of its reduced runtime. However, equation 6.53 is also a valid implementation of the chain rule, and is useful when memory is limited.

applying the back-propagation algorithm to this graph.

Algorithms 6.3 and 6.4 are demonstrations that are chosen to be simple and straightforward to understand. However, they are specialized to one specific problem.

Modern software implementations are based on the generalized form of back-propagation described in section 6.5.6 below, which can accommodate any computational graph by explicitly manipulating a data structure for representing symbolic computation.

Algorithm 6.3 Forward propagation through a typical deep neural network and the computation of the cost function. The loss $L(\hat{\mathbf{y}}, \mathbf{y})$ depends on the output $\hat{\mathbf{y}}$ and on the target $\mathbf{y}$ (see section 6.2.1.1 for examples of loss functions). To obtain the total cost J , the loss may be added to a regularizer $\Omega(\theta)$, where θ contains all the parameters (weights and biases). Algorithm 6.4 shows how to compute gradients of J with respect to parameters $\mathbf{W}$ and $\mathbf{b}$. For simplicity, this demonstration uses only a single input example $\mathbf{x}$. Practical applications should use a minibatch. See section 6.5.7 for a more realistic demonstration.

Require: Network depth, l

Require: $\mathbf{W}^{(i)}, i \in \{1, \dots, l\}$, the weight matrices of the model

Require: $\mathbf{b}^{(i)}, i \in \{1, \dots, l\}$, the bias parameters of the model

Require: $\mathbf{x}$, the input to process

Require: $\mathbf{y}$, the target output

$\mathbf{h}^{(0)} = \mathbf{x}$

for $k = 1, \dots, l$ **do**

$\mathbf{a}^{(k)} = \mathbf{b}^{(k)} + \mathbf{W}^{(k)} \mathbf{h}^{(k-1)}$

$\mathbf{h}^{(k)} = f(\mathbf{a}^{(k)})$

end for

$\hat{\mathbf{y}} = \mathbf{h}^{(l)}$

$J = L(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \Omega(\theta)$

6.5.5 Symbol-to-Symbol Derivatives

Algebraic expressions and computational graphs both operate on **symbols**, or variables that do not have specific values. These algebraic and graph-based representations are called **symbolic** representations. When we actually use or train a neural network, we must assign specific values to these symbols. We replace a symbolic input to the network $\mathbf{x}$ with a specific **numeric** value, such as $[1.2, 3.765, -1.8]^\top$.